

Cut Rules in DAG-Based BFT Mempools: Comparing Relaxed Mempool and Autobahn

Woochang Jeong
POSTECH
South Korea
wcjeong@postech.ac.kr

Chanik Park
POSTECH
South Korea
cipark@postech.ac.kr

ABSTRACT

DAG-based BFT protocols decouple transaction dissemination from ordering. A critical but under-examined design choice is the *cut rule*: how the leader selects disseminated data for each commit. We compare two strategies: the *conservative cut* of Relaxed Mempool (top $2f+1$ validators, uniform $H_{\min}$) and the *inclusive cut* of Autobahn (all n lane tips). Through experiments on 10 validators, we show the conservative cut achieves 42% lower latency under stragglers by excluding slow nodes, while the inclusive cut recovers $10\times$ faster from network blips by committing the entire backlog in one slot. These results suggest adaptive cut strategies as a promising direction.

1 INTRODUCTION

DAG-based BFT protocols [1–3] achieve high throughput by separating data dissemination from ordering. Validators broadcast batches into a DAG; a consensus layer periodically *cuts* the DAG to finalize transactions. While consensus algorithms have been extensively studied, the **cut rule**—how the leader decides *which* portion of the DAG to commit—has received less attention. Yet this choice determines straggler resilience, transaction fairness, and post-blip recovery speed (the *hangover effect* [3]).

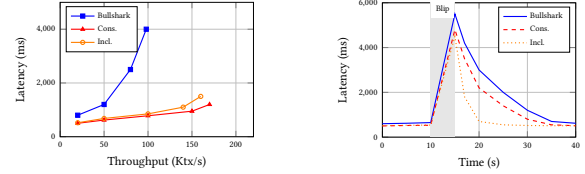
We compare two cut strategies from Relaxed Mempool [4] and Autobahn [3], implement both on a shared Narwhal-based framework, and evaluate their tradeoffs experimentally.

2 CUT RULE DESIGN SPACE

Both systems use **weak quorums** ($f+1$) for speculative data dissemination. The key difference is how the leader constructs commit proposals.

Conservative Cut (Relaxed Mempool). Each block carries a *stream height list*—a vector of known heights per validator. The leader: (1) sorts validators by height, (2) selects the top $2f+1$, (3) computes $H_{\min} = \min$ of their heights, and (4) commits all blocks up to $H_{\min}$. This **uniform cut** excludes the bottom f validators.

Inclusive Cut (Autobahn). Each validator runs an independent *lane*. The leader proposes a vector of n certified tip references—one per lane. Lane coverage requires $\geq n-f$ new tips. After commit, replicas sync missing data and interleave



(a) 4 stragglers (+300 ms)

(b) 5s blip recovery

Figure 1: Performance under (a) straggler and (b) blip conditions.

lanes. This **per-lane cut** includes all validators at their own position.

Table 1: Cut rule comparison.

	Relaxed Mempool	Autobahn
Cut scope	Top $2f+1$	All n lanes
Slow nodes	Excluded	Included
Cut shape	Uniform $H_{\min}$	Per-lane
Data sync	Before finalization	After vote

3 EXPERIMENTAL EVALUATION

We implement both strategies on a modified Narwhal codebase with Bullshark as baseline. Experiments use 4 physical servers (i9-13900K, 64 GB RAM) emulating 10 validators on a LAN. Transactions: 512 B; batch size: 500 KB; batch timeout: 200 ms.

Straggler scenario (Fig. 1a): With 4 stragglers (+300 ms delay), Bullshark drops to 98K tx/s at 4 s latency. The conservative cut reaches 170K tx/s at 1.2 s by excluding slow validators from $H_{\min}$. The inclusive cut achieves 160K tx/s at 1.5 s—slightly worse because referencing all lanes delays data sync.

Hangover effect (Fig. 1b): After a 5s network blip, recovery differs dramatically. Bullshark and the conservative cut need 25s and 20s respectively— $H_{\min}$ advances gradually as slow nodes catch up. The inclusive cut recovers in ~ 2 s: one consensus slot commits all accumulated tips with $O(1)$ complexity, confirming Autobahn’s seamlessness property.

Happy path: Without stragglers, all strategies perform comparably (~ 195 – 205 K tx/s, ~ 500 ms latency), confirming the cut rule primarily matters under adversity.

4 DISCUSSION

Neither cut strategy dominates universally. The conservative cut excels under persistent stragglers; the inclusive cut

excels in blip-prone environments. A natural extension is an **adaptive cut** that monitors validator height disparity to dynamically switch modes. Both strategies share the same DAG substrate, making runtime switching feasible with minimal overhead.

REFERENCES

- [1] G. Danezis et al., “Narwhal and Tusk,” *EuroSys*, 2022.
- [2] A. Spiegelman et al., “Bullshark,” *ACM CCS*, 2022.
- [3] N. Girdharan et al., “Autobahn: Seamless high speed BFT,” *SOSP*, 2024.
- [4] W. Jeong and C. Park, “Breaking the straggler effect: A Relaxed Mem-pool for DAG-based BFT,” 2025.